



Virtual Garage Full Stack API - Exam Study Guide

Table of Contents

1. [React Components & Lifecycle](#)
 2. [React Hooks Deep Dive](#)
 3. [Component Rendering & Re-rendering](#)
 4. [Stripe Payment Integration](#)
 5. [IdentityServer Authentication & Authorization](#)
 6. [Architecture Overview](#)
-

React Components & Lifecycle

What is a React Component?

A React component is a reusable piece of UI that can:

- Accept inputs (props)
- Manage its own state
- Render JSX (HTML-like syntax)
- React to changes and re-render when state or props change

Component Lifecycle Phases

1. **Mounting**: Component is created and inserted into the DOM
 2. **Updating**: Component re-renders due to state/props changes
 3. **Unmounting**: Component is removed from the DOM
-

React Hooks Deep Dive

1. useState Hook

Purpose: Adds state management to functional components

Syntax:

```
const [state, setState] = useState(initialValue);
```

Example from VehiclesPage:

```
const [vehicles, setVehicles] = useState<Vehicle[]>([]);  
const [loading, setLoading] = useState(true);  
const [isAdding, setIsAdding] = useState(false);
```

How it works:

- Returns an array with 2 elements: current state value and updater function
- When you call `useState`, React schedules a re-render
- React preserves state between re-renders
- Each `useState` call creates an independent state variable

Re-render triggers:

- ☒ Calling `setVehicles([...])` triggers re-render
- ☒ Calling `setLoading(false)` triggers re-render
- ☒ Directly mutating `vehicles.push()` does NOT trigger re-render (wrong!)

Real example - Adding a vehicle:

```
// ✗ WRONG - mutates state directly, no re-render
vehicles.push(newVehicle);

// ✓ CORRECT - creates new array, triggers re-render
setVehicles((vs) => [...vs, newVehicle]);
```

2. useEffect Hook

Purpose: Performs side effects in functional components (data fetching, subscriptions, DOM manipulation)

Syntax:

```
useEffect(() => {
  // Effect logic
  return () => {
    // Cleanup (optional)
  };
}, [dependencies]);
```

Example from VehiclesPage:

```
useEffect(() => {
  getMyVehicles().then((v) => {
    setVehicles(v);
    setLoading(false);
  });
}, []); // Empty array = runs once on mount
```

Dependency Array Rules:

Dependency Array	When Effect Runs
------------------	------------------

Dependency Array	When Effect Runs
<code>[]</code>	Only once when component mounts
<code>[count]</code>	On mount + every time <code>count</code> changes
<code>[user, vehicles]</code>	On mount + when <code>user</code> OR <code>vehicles</code> change
No array (omitted)	On mount + after EVERY render (dangerous!)

Example from DonationPage:

```
// In DonationPage, email is prefilled from auth:
const [email, setEmail] = useState(auth?.user?.profile?.email || "");
```

No `useEffect` needed here because:

- Initial value is set once during component creation
- Email doesn't need to sync with auth changes (it's user-editable)

When you `WOULD` use `useEffect`:

```
// If email should always stay synced with auth:
useEffect(() => {
  if (auth?.user?.profile?.email) {
    setEmail(auth.user.profile.email);
  }
}, [auth?.user?.profile?.email]); // Re-sync when auth email changes
```

Cleanup Function Example:

```
useEffect(() => {
  const url = URL.createObjectURL(blob);
  setPdfUrl(url);

  return () => {
    // Cleanup: release memory when component unmounts
    URL.revokeObjectURL(url);
  };
}, [blob]);
```

3. Custom Hooks

Purpose: Extract reusable logic from components

Example - `useAuthContext` (from your codebase):

```
export const useAuthContext = () => {
  const auth = useAuth(); // Uses react-oidc-context

  const isAuthenticated = auth.isAuthenticated || false;
  const user = auth.user;
  const accessToken = user?.access_token;

  // Extract roles from JWT token
  const roles: string[] = (() => {
    if (!user?.profile?.role) return [];
    const roleClaim = user.profile.role;
    return Array.isArray(roleClaim) ? roleClaim : [roleClaim];
  })();

  const isAdmin = roles.includes("Admin");

  return {
    auth,
    isAuthenticated,
    user,
    accessToken,
    roles,
    isAdmin,
    isLoading: auth.isLoading,
    error: auth.error,
  };
};
```

Usage in a component:

```
function AdminDashboardPage() {
  const { isAdmin, isAuthenticated } = useAuthContext();

  if (!isAuthenticated) return <Navigate to="/login" />;
  if (!isAdmin) return <p>Access denied</p>;

  return <div>Admin Dashboard</div>;
}
```

Why custom hooks?

- ☒ Reusable across multiple components
- ☒ Encapsulates complex logic
- ☒ Easier to test
- ☒ Cleaner component code

Component Rendering & Re-rendering

What Causes a Re-render?

1. State Changes (`useState`)

```
const [count, setCount] = useState(0);

// This triggers re-render:
setCount(count + 1);

// Parent re-renders → Child re-renders too
```

2. Props Changes

```
// Parent component
function VehiclesPage() {
  const [vehicles, setVehicles] = useState([]);

  return <VehicleCard vehicle={vehicles[0]} />; // ← Props
}

// Child component re-renders when vehicle prop changes
function VehicleCard({ vehicle }: Props) {
  return <div>{vehicle.brand}</div>;
}
```

3. Parent Component Re-renders

Important: When a parent re-renders, all children re-render by default!

```
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>Click</button>
      <Child /> { /* ← Re-renders even if it doesn't use count! */ }
    </>
  );
}
```

Real Example: VehiclesPage Re-render Chain

Initial Load:

1. Component mounts
2. `useEffect` runs (dependency array `[]`)

3. `getMyVehicles()` API call
4. `setVehicles(v)` triggers re-render
5. `setLoading(false)` triggers another re-render
6. All `VehicleCard` children re-render

Adding a Vehicle:

1. User clicks "Add Vehicle"
2. `setIsAdding(true)` → Re-render (modal appears)
3. User submits form
4. `onCreated` callback: `setVehicles(vs => [...vs, v])` → Re-render
5. `setIsAdding(false)` → Re-render (modal closes)
6. All `VehicleCard` components re-render (including the new one)

Deleting a Vehicle:

```
async function confirmDelete() {
  if (!vehicleToDelete) return;

  await deleteVehicle(vehicleToDelete.id);

  // This triggers re-render:
  setVehicles((vs) => vs.filter((v) => v.id !== vehicleToDelete.id));

  closeDelete(); // setVehicleToDelete(null) → Another re-render
}
```

State Updates Cause:

- ☒ `setVehicles()` → VehiclesPage re-renders
- ☒ All VehicleCard children re-render
- ☒ Modal state changes (`setIsAdding`, `setVehicleToDelete`) → Re-render

Re-render Optimization (Not implemented in your code, but good to know)

useMemo: Memoize expensive calculations

```
const expensiveValue = useMemo(() => {
  return vehicles.filter((v) => v.year > 2020).length;
}, [vehicles]); // Only recalculates when vehicles change
```

useCallback: Memoize functions to prevent child re-renders

```
const handleDelete = useCallback((id: string) => {
  deleteVehicle(id);
}, []);
```

React.memo: Prevent child re-render if props haven't changed

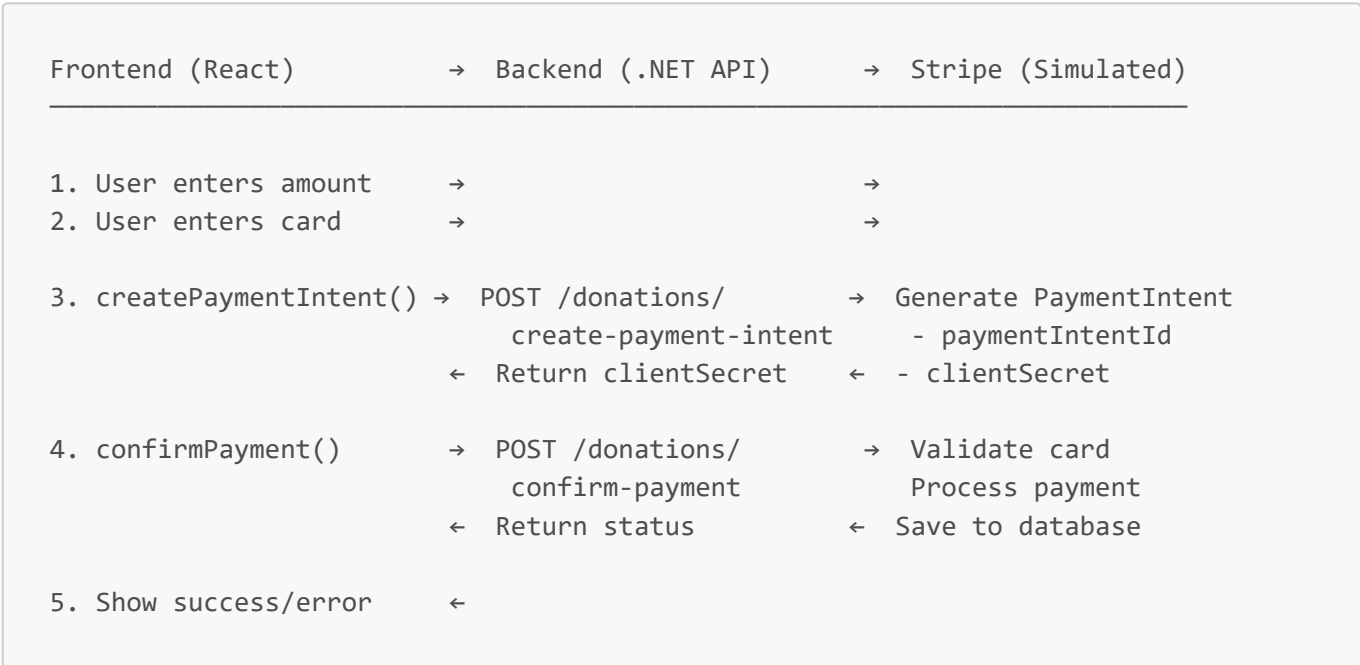
```
const VehicleCard = React.memo(({ vehicle }: Props) => {
  return <div>{vehicle.brand}</div>;
});
```

Stripe Payment Integration

Overview

Your app uses a **simulated Stripe payment flow** (not real Stripe API, but mimics the pattern).

Payment Flow Architecture



Frontend: Creating Payment Intent

File: `src/api/donations.ts`

```
export async function createPaymentIntent(
  data: CreatePaymentIntentRequest,
): Promise<CreatePaymentIntentResponse> {
  const response = await api.post<CreatePaymentIntentResponse>(
    "/donations/create-payment-intent",
    data,
  );
  return response.data;
}
```

DonationPage Usage:

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setLoading(true);

  // Step 1: Create payment intent
  const { clientSecret, paymentIntentId } = await createPaymentIntent({
    amount: Math.round(finalAmount * 100), // Convert dollars to cents!
    email,
    currency: "usd",
  });

  // Step 2: Confirm payment (simulated card processing)
  await confirmPayment({
    paymentIntentId,
    clientSecret,
    card: {
      number: cardNumber.replace(/\s/g, ""),
      exp_month: cardExpiry.split("/")[0],
      exp_year: cardExpiry.split("/")[1],
      cvc: cardCvc,
    },
  });

  setMessage({ type: "success", text: "Payment successful!" });
  setLoading(false);
};

```

Backend: Processing Payment

File: `VirtualGarage.Domain.Services/DonationService.cs`

Creating Payment Intent:

```

public async Task<CreatePaymentIntentResponse> CreatePaymentIntentAsync(
    CreatePaymentIntentRequest request,
    Guid userId
)
{
    // Validate input
    if (request.Amount <= 0)
        throw new ArgumentException("Amount must be greater than 0");

    // Generate payment intent ID (simulated)
    string paymentIntentId = $"pi_{Guid.NewGuid().ToString("N").Substring(0,
20)}";
    string clientSecret = $"pi_{...}_secret_{...}";

    // Store temporarily (in production, Stripe stores this)
    PaymentIntents[paymentIntentId] = new PaymentIntentData(
        clientSecret, request.Amount, request.Currency,

```

```
        request.Email, userId
    );

    return new CreatePaymentIntentResponse {
        PaymentIntentId = paymentIntentId,
        ClientSecret = clientSecret,
        Amount = request.Amount,
        Currency = request.Currency,
    };
}
```

Confirming Payment:

```
public async Task<ConfirmPaymentResponse> ConfirmPaymentAsync(
    ConfirmPaymentRequest request,
    Guid userId
)
{
    // Validate that payment intent exists
    if (!PaymentIntents.TryGetValue(request.PaymentIntentId, out var intentData))
        throw new InvalidOperationException("Payment intent not found");

    // Verify client secret matches
    if (intentData.ClientSecret != request.ClientSecret)
        throw new InvalidOperationException("Invalid client secret");

    // Simulate card validation (test card: 4242 4242 4242 4242)
    string cardNumber = request.Card.Number;
    if (cardNumber.Length < 13)
        throw new InvalidOperationException("Invalid card number");

    bool isTestCard = cardNumber.StartsWith("4242");

    // Save donation record to database
    var donation = new Donation {
        UserId = userId,
        Amount = intentData.Amount,
        Currency = intentData.Currency,
        Email = intentData.Email,
        PaymentIntentId = request.PaymentIntentId,
        Status = "succeeded",
    };

    await donationRepository.CreateDonationAsync(donation);

    return new ConfirmPaymentResponse {
        Status = "succeeded",
        PaymentIntentId = request.PaymentIntentId,
        Amount = intentData.Amount,
    };
}
```

Key Stripe Concepts

1. Payment Intent

- Represents a payment in progress
- Created before card details are collected
- Tracks payment lifecycle (pending → succeeded/failed)

2. Client Secret

- Secure token used to confirm payment from frontend
- Should be kept private (not shared publicly)
- Each payment intent has unique client secret

3. Amount in Cents

Important: Always store amounts in smallest currency unit (cents for USD)

```
// User enters: $25.50
const dollars = 25.5;
const cents = Math.round(dollars * 100); // 2550 cents

// Backend stores: 2550
// Display: formatAmount(2550, "usd") → "$25.50"
```

4. Test Cards (Real Stripe)

Card Number	Result
4242 4242 4242 4242	Success
4000 0000 0000 0002	Card declined
4000 0000 0000 9995	Insufficient funds

Your app accepts any card starting with "4242" or valid card formats.

IdentityServer Authentication & Authorization

Overview Architecture



2. API request	→	Validates JWT token
+ Bearer token		Checks scopes & roles
	←	Returns data

Authentication Flow (OpenID Connect)

1. Configuration (Frontend)

File: `src/config/oidcConfig.ts`

```
export const oidcConfig: AuthProviderProps = {
  authority: "https://localhost:5001", // IdentityServer URL
  client_id: "react-app-client",
  client_secret: "reactapp-secret",
  redirect_uri: "http://localhost:5173/callback",
  scope: "openid profile roles virtualgarage.api.read virtualgarage.api.write",
  response_type: "code", // Authorization Code Flow
  post_logout_redirect_uri: "http://localhost:5173/login",
};
```

Key Scopes:

- `openid`: Required for OpenID Connect
- `profile`: Access to user profile info (name, email)
- `roles`: Access to user roles (Admin, User)
- `virtualgarage.api.read`: Permission to read data from API
- `virtualgarage.api.write`: Permission to write data to API

2. IdentityServer Configuration (Backend)

File: `VirtualGarage.IdentityServer/Config.cs`

Identity Resources (User info):

```
public static IEnumerable<IdentityResource> IdentityResources =>
    new IdentityResource[]
    {
        new IdentityResources.OpenId(), // Required
        new IdentityResources.Profile(), // Name, email, etc.
        new IdentityResource("roles", new[] { "role" }) // Custom: roles
    };
```

API Resources (Protected APIs):

```
public static IEnumerable<ApiResource> ApiResources =>
    new ApiResource[]
```

```
{
    new ApiResource("virtualgarage.api", "Virtual Garage API")
    {
        Scopes = {
            "virtualgarage.api.read",
            "virtualgarage.api.write"
        },
        UserClaims = { "role" } // Include role in token
    }
};
```

Client Configuration:

```
new Client
{
    ClientId = "react-app-client",
    ClientSecrets = { new Secret("reactapp-secret".Sha256()) },
    AllowedGrantTypes = GrantTypes.Code, // Authorization Code Flow

    AllowedScopes = {
        IdentityServerConstants.StandardScopes.OpenId,
        IdentityServerConstants.StandardScopes.Profile,
        "roles",
        "virtualgarage.api.read",
        "virtualgarage.api.write"
    },

    RedirectUri = { "http://localhost:5173/callback" },
    PostLogoutRedirectUri = { "http://localhost:5173/login" },
    AllowedCorsOrigins = { "http://localhost:5173" },
}
```

3. Login Flow Step-by-Step

Step 1: User clicks "Login"

```
const auth = useAuth();
auth.signinRedirect();
```

Step 2: User redirected to IdentityServer (<https://localhost:5001>)

- Shows login page
- User enters username/password

Step 3: IdentityServer validates credentials

- Checks against database (ASP.NET Identity)
- Creates authorization code

Step 4: User redirected back to app with code

```
http://localhost:5173/callback?code=ABC123...
```

Step 5: App exchanges code for tokens

```
// Automatically handled by react-oidc-context
// Returns:
{
  access_token: "eyJhbGc...", // JWT token for API
  id_token: "eyJhbGc...",     // User identity info
  refresh_token: "...",       // For refreshing access token
}
```

Step 6: App stores tokens and user info

```
const { isAuthenticated, user, accessToken } = useAuthContext();

// user.profile contains:
{
  sub: "123-456-789", // User ID
  name: "John Doe",
  email: "john@example.com",
  role: ["User", "Admin"]
}
```

Authorization (Roles & Scopes)

Frontend: Checking Permissions**Custom Hook:**

```
const { isAdmin, isAuthenticated, roles } = useAuthContext();

if (!isAuthenticated) {
  return <Navigate to="/login" />;
}

if (!isAdmin) {
  return <p>Access denied</p>;
}
```

Backend: API Protection

File: `VirtualGarage.Api/Program.cs`

JWT Authentication Setup:

```
builder.Services.AddAuthentication(options => {
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options => {
    options.Authority = "https://localhost:5001"; // IdentityServer
    options.TokenValidationParameters.ValidateAudience = false;
    options.TokenValidationParameters.RoleClaimType = "role";
    options.SaveToken = true;
});
```

Authorization Policies:

```
builder.Services.AddAuthorization(options =>
{
    // Simple role-based policy
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));

    // Custom policy: requires scope OR role
    options.AddPolicy("VehicleReadPolicy", policy =>
        policy.Requirements.Add(
            new ClaimOrRoleRequirement(
                "virtualgarage.api.read", // Scope
                "User" // OR Role
            )
        ));
});
```

Custom Authorization Handler:

File: `VirtualGarage.Api/VirtualGarageAuthHandler.cs`

```
public class VirtualGarageAuthHandler :
    AuthorizationHandler<ClaimOrRoleRequirement>
{
    protected override async Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        ClaimOrRoleRequirement requirement)
    {
        var claims = context.User.Claims.ToList();

        // Check if user has required scope
        var hasScope = claims.Exists(c => c.Value == requirement.Claim) ||
            claims.Exists(c => c.Type == "scope" &&
```

```

        c.Value.Split('
    ').Contains(requirement.Claim));

    if (!hasScope)
    {
        context.Fail(); // Unauthorized
        return;
    }

    // Additional checks for user-specific authorization
    var userId = claims.FirstOrDefault(c => c.Type == "sub")?.Value;

    if (userId is not null)
    {
        // Fetch user info from IdentityServer
        var disco = await
client.GetDiscoveryDocumentAsync("https://localhost:5001");
        var userInfoResponse = await client.GetUserInfoAsync(new
UserInfoRequest {
            Address = disco.UserInfoEndpoint,
            Token = accessToken
        });

        var userRoles = userInfoResponse.Claims
            .Where(c => c.Type == "role")
            .Select(c => c.Value)
            .ToList();

        // Check if user has required role
        if (userRoles.Contains(requirement.Role))
        {
            context.Succeed(requirement); // Authorized
        }
        else
        {
            context.Fail(); // Forbidden
        }
    }
}
}
}

```

Controller Protection

```

[Authorize] // Requires authentication
[ApiController]
[Route("api/[controller]")]
public class VehiclesController : ControllerBase
{
    [HttpGet]
    [Authorize(Policy = "VehicleReadPolicy")] // Requires scope/role
    public async Task<ActionResult<List<VehicleDto>>> GetMyVehicles()

```

```

    {
        var userId = User.FindFirstValue("sub"); // Get user ID from JWT
        // ...
    }

    [HttpDelete("{id}")]
    [Authorize(Policy = "VehicleWritePolicy")]
    public async Task<IActionResult> DeleteVehicle(Guid id)
    {
        // ...
    }

    [HttpGet("all")]
    [Authorize(Policy = "AdminOnly")] // Only admins
    public async Task<ActionResult<List<VehicleDto>>> GetAllVehicles()
    {
        // ...
    }
}

```

JWT Token Structure

Example Access Token (decoded):

Header:

```

{
  "alg": "RS256",
  "typ": "JWT"
}

```

Payload (claims):

```

{
  "sub": "123-456-789", // User ID
  "name": "John Doe",
  "email": "john@example.com",
  "role": ["User", "Admin"],
  "scope": "openid profile roles virtualgarage.api.read virtualgarage.api.write",
  "iss": "https://localhost:5001", // Issuer (IdentityServer)
  "aud": "virtualgarage.api", // Audience (API)
  "exp": 1735689600, // Expiration timestamp
  "iat": 1735686000 // Issued at timestamp
}

```

Signature: Cryptographic signature to verify token hasn't been tampered with

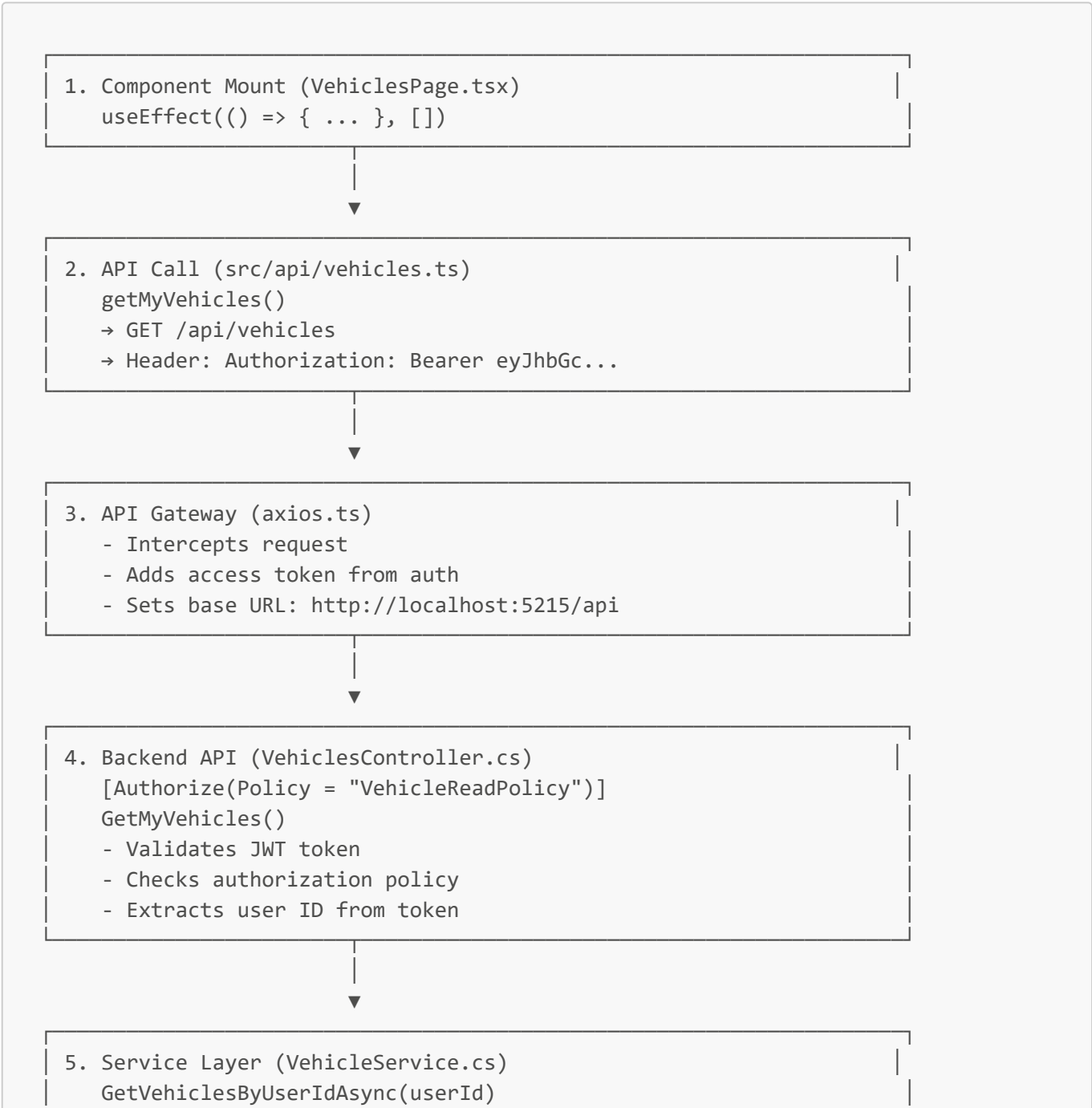
Authentication vs Authorization Summary

Aspect	Authentication	Authorization
Question	Who are you?	What can you do?
Process	Login with username/password	Check roles/scopes in JWT
Result	JWT access token	Allow/deny API access
Frontend	<code>isAuthenticated</code>	<code>isAdmin, roles</code>
Backend	<code>[Authorize]</code>	<code>[Authorize(Policy = "...")]</code>

Architecture Overview

Frontend → Backend Flow

Example: Loading User's Vehicles



- Business logic
- Calls repository



6. Repository (VehicleRepository.cs)
- GetVehiclesByUserIdAsync(userId)
- Queries database using Entity Framework
 - Returns Vehicle entities



7. Database (SQL Server)
- SELECT * FROM Vehicles WHERE UserId = @userId



8. Response Flows Back
- Repository → Service → Controller → JSON → Frontend



9. Update State & Re-render (VehiclesPage.tsx)
- setVehicles(v) → triggers re-render
- setLoading(false) → triggers re-render
- All VehicleCard components render with data

Project Structure

```
VirtualGarage/
├── virtualgarage-frontend/      # React SPA
│   ├── src/
│   │   ├── api/                # API client functions
│   │   ├── components/         # Reusable UI components
│   │   ├── config/             # OIDC configuration
│   │   ├── hooks/              # Custom hooks (useAuthContext)
│   │   ├── pages/              # Page components
│   │   ├── providers/          # Context providers
│   │   └── types/              # TypeScript types
│   └── .env.production          # Production environment variables
├── VirtualGarage.Api/          # Web API (.NET)
│   ├── Controllers/            # API endpoints
│   ├── Program.cs              # DI, auth, CORS setup
│   └── VirtualGarageAuthHandler.cs # Custom authorization
```

```
├─ VirtualGarage.IdentityServer/ # Authentication server
│   ├── Config.cs                # Clients, scopes, resources
│   ├── Data/                    # User database (ASP.NET Identity)
│   └── Pages/                   # Login/Register UI
├─ VirtualGarage.Domain.Services/ # Business logic
│   ├── DonationService.cs
│   ├── UserService.cs
│   └── VehicleService.cs
├─ VirtualGarage.Persistence/    # Data access
│   ├── DbContexts/
│   ├── Repositories/
│   └── Migrations/
└─ VirtualGarage.Domain/         # Domain models
    └─ VehicleSpecs.cs
```

Quick Reference

Re-render Causes

1. ☒ `useState` setter called
2. ☒ Props changed from parent
3. ☒ Parent component re-rendered
4. ☒ `useContext` value changed
5. ☒ Direct state mutation (e.g., `array.push()`)

useState Best Practices

- ☒ Use functional updates: `setState(prev => prev + 1)`
- ☒ Create new arrays/objects: `setState([...old, new])`
- ☒ Never mutate state directly: `state.push()`, `state.x = 5`

useEffect Patterns

```
// Run once on mount
useEffect(() => {}, []);

// Run when dependencies change
useEffect(() => {}, [count, user]);

// Cleanup on unmount
useEffect(() => {
  return () => {
    /* cleanup */
  };
}, []);
```

Authentication Flow

1. User logs in → IdentityServer
2. Returns JWT access token
3. Frontend stores token
4. Adds token to API requests: `Authorization: Bearer <token>`
5. Backend validates token & checks permissions

Authorization Hierarchy

- **Authentication:** Requires valid JWT token (`[Authorize]`)
 - **Scope:** Requires specific API scope claim (e.g., `virtualgarage.api.read`)
 - **Role:** Requires specific role (e.g., `Admin`)
 - **Policy:** Combines scopes + roles (`[Authorize(Policy = "...")]`)
-

Exam Tips

React Concepts

- **useState:** Creates state, triggers re-render when changed
- **useEffect:** Runs side effects, controlled by dependency array
- **Custom hooks:** Reusable logic extraction, must start with "use"
- **Re-rendering:** Happens when state/props change or parent re-renders

Stripe Flow

1. Create payment intent (get clientSecret)
2. Collect card details
3. Confirm payment (validate card, process)
4. Save donation record to database

IdentityServer

- **Authentication:** Who you are (JWT token)
- **Authorization:** What you can do (roles, scopes, policies)
- **Scopes:** Permissions for APIs (`virtualgarage.api.read`)
- **Roles:** User types (`Admin`, `User`)
- **JWT:** Contains user ID (sub), name, email, roles, scopes

Good luck on your exam! 🚀